Chapter 13: Moving to MongoDB Atlas

Our Pets application already uses documents. It knows how to find an owner, save a pet, and return dictionaries to Flask. Now we will move storage out of the Python process and connect to MongoDB running on a server.

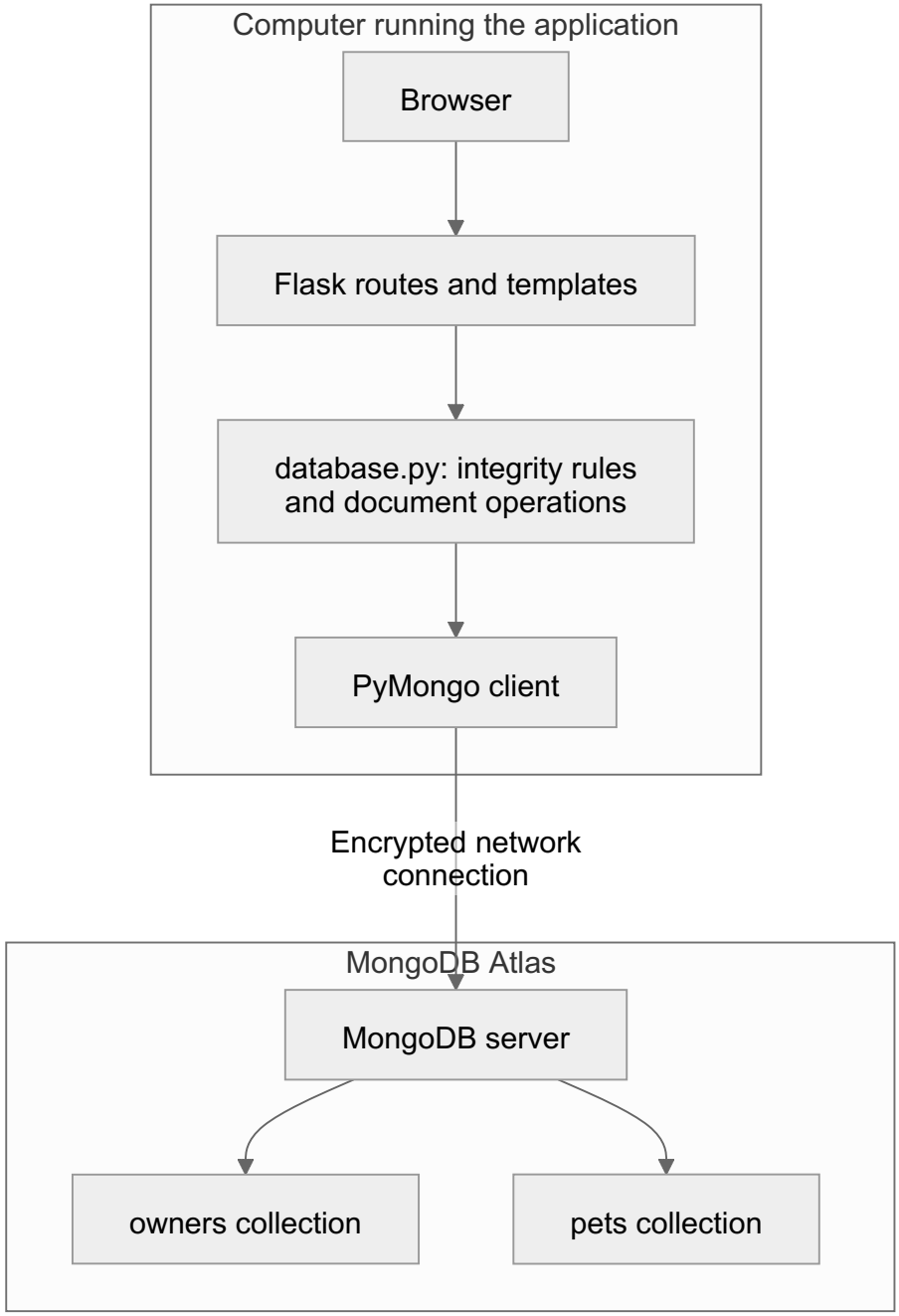
MongoDB Atlas is a managed cloud database service. We choose a deployment and configure access; the service operates the database servers. Cloud databases can be relational or document-oriented. The term describes where and how the database runs, not how its records must be organized. (1)

The application will still run on our computer. Atlas will hold the documents. Moving the database does not publish the Flask website or upload its Python files.

What Changes, and What Stays the Same

Mongita implements document operations locally. PyMongo is a driver: it sends operations to MongoDB and receives results. The similar interfaces let us keep most of the database layer as it is.

Part	Local Mongita version	Atlas version
Browser and Flask routes	Forms and requests	Same forms and requests
Database functions	`get_pet`, `create_pet`, and the others	Same function interface
Integrity rules	Python checks in `database.py`	Same checks
Client	`MongitaClientDisk`	PyMongo `MongoClient`
Stored documents	Local folder	MongoDB deployment in Atlas
Access	Local files	Network connection and database credentials



The browser talks to Flask. Flask calls `database.py`. Only the database layer needs the database connection settings. A pet's identifier still travels through the application as a string, while its stored `_id` and `owner_id` remain `ObjectId` values.

We will start with fresh records in Atlas. Changing a connection does not copy the documents from the local Mongita folder. Moving existing records is a separate data migration, discussed below.

Set Up Atlas for the First Time

We need an Atlas login, a project, and a database deployment before Python can connect. Use your own account and project for this exercise. Keep these names separate:

Item	Purpose	Our example
Atlas account	Sign in to the management website	Your login
Project	Group deployments and access settings	`Advanced-Database`
Cluster	Run MongoDB and hold databases	`course-pets`
Database	Group our application's collections	`pets`
Database user	Let Python authenticate	`user`

1. Create an Account and Log In

Try This: Open the Atlas website at https://cloud.mongodb.com/. Choose **Sign Up** if you do not already have an account. Register with an email address or one of the offered Google or GitHub sign-in options. Complete any verification prompts, then sign in using the same method. Existing users can log in directly. (2)

Finish the welcome questions. Atlas may create an organization and an initial project for a new account, then take you straight to deployment setup. An organization is the container above projects; the initial organization is sufficient for this exercise. You do not need to create a company or invite classmates. (2)

2. Create or Select the Course Project

Try This: Check the project selector near the top of the Atlas page. If onboarding already created a project for this exercise, use it. Otherwise, open the **Projects** menu and choose **New Project**. Name it `Advanced-Database`, click **Next**, and complete **Create Project**. No additional members are needed. You can also reach **New Project** from the organization's **All Projects** page. (3)

Confirm that the course project is selected before continuing. Database users and network access entries belong to a project, so configuring another project's settings will not configure this one.

3. Create a Free Cluster

A cluster is the MongoDB deployment we connect to. Atlas's Free tier, also called `M0`, is suitable for this small course application. (4)

Try This: From **Project Overview**, click **Create** to open deployment setup. First-time onboarding may already have opened this screen.

1. Select **Free** or **M0**. Confirm the tier before continuing; do not select a paid option for this exercise.
2. Choose an available cloud provider and a nearby region that offers the Free tier. Any offered provider is suitable for our Pets example.
3. Enter `course-pets` as the cluster name. The name cannot be changed after deployment.

4. Turn off **Automate security setup** so we can configure the user and network access ourselves. Leave **Preload sample dataset** off. Click **Create Deployment** to deploy the cluster. Wait until Atlas shows it is ready. (4)

Skip loading sample datasets if offered. We will create our own records. The cluster name `course-pets` is not the database name `pets`; one cluster can hold multiple databases.

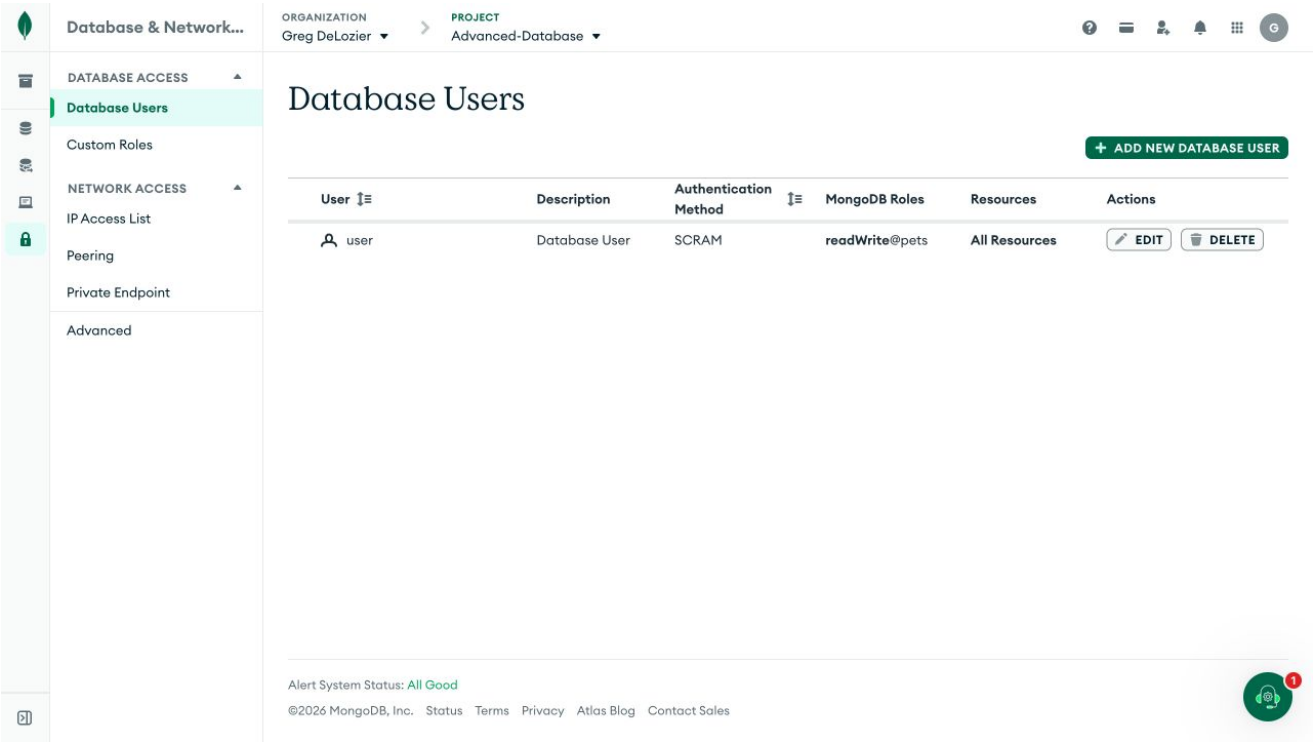
Atlas may now open **Security Quickstart** for a database user and network access. Complete those steps as described below. If they were already completed during onboarding, inspect the existing settings instead of creating duplicate entries.

4. Create the Database User

Your Atlas account opens the management website. A **database user** authenticates an application connection. These are separate identities. The Python program uses the database user's name and password, not your Atlas website password. Database roles determine which operations that user may perform. (5)

Try This: In Security Quickstart, create `user` with password authentication. For our temporary classroom example, use password `alphabeta123`, then click **Create Database User**. If you are outside the wizard, open **Database Access** or **Database Users** in the project's navigation and choose **Add New Database User**.

Open the user's privileges settings (use **Edit** if Quickstart already created it). Under specific privileges, select `readWrite` for database `pets`, applying to all its collections. Remove a broader all-database role if Quickstart assigned one. Save the user. The `pets` database does not have to exist yet. (5)



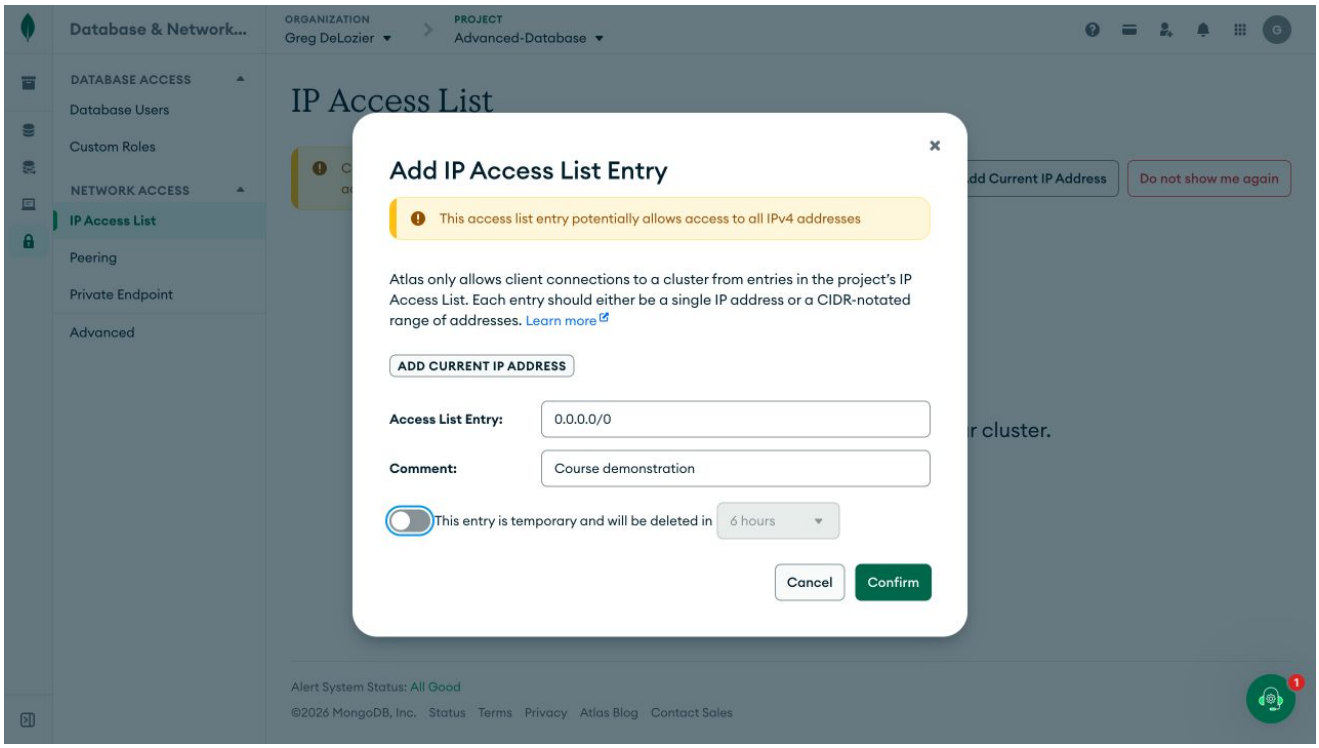
The classroom database user has readWrite access to pets.

There are also two different meanings of "owner" here. Alex is a document in our `owners` collection. Alex is not an Atlas database user. The application connects as `user` while working with Alex's records.

5. Allow Network Connections

Atlas checks an Internet Protocol (IP) access list before accepting connections. For this classroom example, use `0.0.0.0/0`, which permits connections from any IPv4 address. This lets the same example run on a laptop or in a Codespace without updating the list when the network address changes. Database authentication is still required. (6)

Try This: Open **Database & Network Access**, select **IP Access List**, and click **Add IP Address**. Enter `0.0.0.0/0` in **Access List Entry** and `Course demonstration` in **Comment**. Leave the temporary-entry option off, click **Confirm**, and wait for the entry to show **Active**.



Network access for the classroom example. The entry allows any IPv4 address; the database still requires credentials.

Use the complete value, including `/0`. A bare `0.0.0.0` does not specify the same range. Network access and database authentication are separate checks.

6. Get the Python Connection String

Try This: Return to the cluster overview and click **Connect** beside `course-pets`. Complete any outstanding connection-security prompts, then choose **Drivers** or the client-library connection option. Select **Python** and **PyMongo (Synchronous)**. Copy the supplied connection URI. (7)

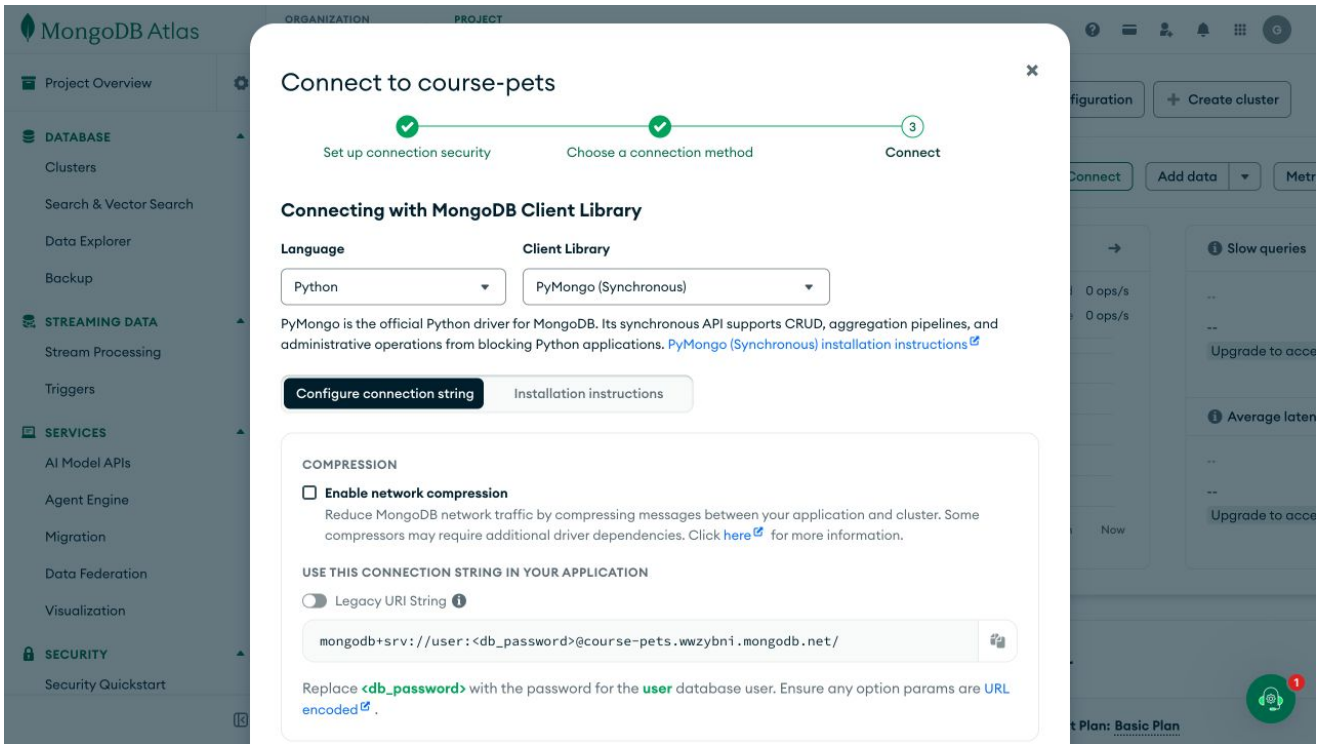
Replace the username and password placeholders with `user` and its database password. Remove the placeholder angle brackets. Password characters with special URI meanings must be percent-encoded; follow the instructions alongside the string. Atlas connections use Transport Layer Security (TLS) to encrypt traffic. (7)

At this point, the cluster should be ready, `user` should be listed as a database user, and the application's address should appear in the IP access list. Keep the connection string available for the terminal steps below.

A connection string, also called a Uniform Resource Identifier (URI), includes the server address and login credentials. The following string records the temporary deployment used for this walkthrough. **That deployment and its database user have been deleted.** Create your own cluster using the steps above and substitute its hostname and credentials in every command below:

```
mongodb+srv://user:alphabeta123@course-pets.wwzybni.mongodb.net/
```

The username is `user` and the password is `alphabeta123`. These credentials are deliberately shared for this short-lived student exercise. **Do not leave these shared credentials or the open `0.0.0.0/0` network entry active in service after the exercise.** When finished, delete the temporary database user and remove the open network entry, or terminate the exercise cluster. Use only exercise data in this deployment; it will not remain available indefinitely. When creating your own cluster, use its hostname in place of `course-pets.wwzybni.mongodb.net`.



Select Python and PyMongo (Synchronous). Atlas displays a password placeholder; substitute the classroom password in the connection string.

The code selects the `pets` database explicitly. Our program reads the connection string from an environment variable named `MONGODB_URI`, so we can point the same code at a different deployment without editing the database layer.

Prepare Python and Set the Environment

Use Python 3.10 or newer. From the repository root, create a virtual environment, activate it, and install the chapter's dependencies. These commands use a Bash terminal on Linux or macOS:

```
cd chapter-13-mongo-atlas/code
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
```

In Windows PowerShell, the corresponding commands are:

```
cd chapter-13-mongo-atlas/code
py -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
```

The requirements install Flask and PyMongo. Mongita is no longer needed to run this application. It remains available in the separate test requirements as a local memory store for tests.

Try This: In Bash, set the connection string, replacing the retired example with your own deployment:

```
export MONGODB_URI='mongodb+srv://user:alphabeta123@course-pets.wwzybni.mongodb.net/'
```

In Windows PowerShell:

```
$env:MONGODB_URI = 'mongodb+srv://user:alphabeta123@course-pets.wwzybni.mongodb.net/'
```

Run the following commands in that same terminal. A new terminal will need its own setting. This application does not automatically read a `.env` file.

Check the Connection Before Starting Flask

Try This: Run the supplied connection demonstration:

```
python access-atlas-demo.py
```

On a fresh database, the expected output is:

```
Connected to MongoDB; pets database is readable.
Owners: 0
Pets: 0
```

Existing records produce larger counts. The demonstration does not delete or insert anything. It calls `setup_database("pets")`, reads the two counts, and closes the connection.

Inside `database.py`, the normal connection path reads the environment variable and creates the client:

```
uri = os.environ.get("MONGODB_URI")
if not uri:
    raise RuntimeError("Set MONGODB_URI before starting the application.")
client = MongoClient(
    uri, server_api=ServerApi("1"), serverSelectionTimeoutMS=10000
)
```

`ServerApi("1")` selects MongoDB's Stable application programming interface (API), which helps keep supported command behavior consistent as server versions change. The client then runs `client.admin.command("ping")` to make a connection attempt immediately. Constructing a client alone is not our connection check. (8)

The ten-second server-selection setting limits how long the driver waits to find a suitable server. It is not a ten-second limit on every query. After connecting, the layer selects the database and collections:

```
db = client[database_name]
owners_collection = db.owners
pets_collection = db.pets
```

These assignments create Python handles. They do not insert documents or create visible stored collections by themselves. MongoDB creates a missing collection on the first insert unless it is explicitly created earlier. (9)

The client stays available for later requests. We do not open a new client inside every `get_pet()` or `create_owner()` call. The connection demonstration closes it because that program's work is finished; the Flask application keeps using it.

Run the Pets Application

Try This: Start Flask from the `code` folder, in the terminal where `MONGODB_URI` is set:

```
flask --app app run --debug
```

Open the local address Flask prints. Create an owner named Alex, with city Kent and home type house. Create Casey, a three-year-old dog, and choose Alex as the owner. The list should show Casey and Alex's identifier.

Open the deployment's **Data Explorer** in Atlas and refresh the collection list. Select the `pets` database. You should find an Alex document in `owners` and a Casey document in `pets`. Data Explorer is another way to inspect the same stored documents. (9)

Compare the owner's `_id` with Casey's `owner_id`. They should be equal `ObjectId` values. The application displays a string version of that identifier, but the reference stored in MongoDB retains its type.

Try This: Change Casey's age to four through the web form. Refresh the document in Data Explorer and check the age. Stop Flask, start it again, and open the list. Casey should still be there with age four. The records belong to the Atlas deployment, not the Flask process.

The create function still looks familiar:

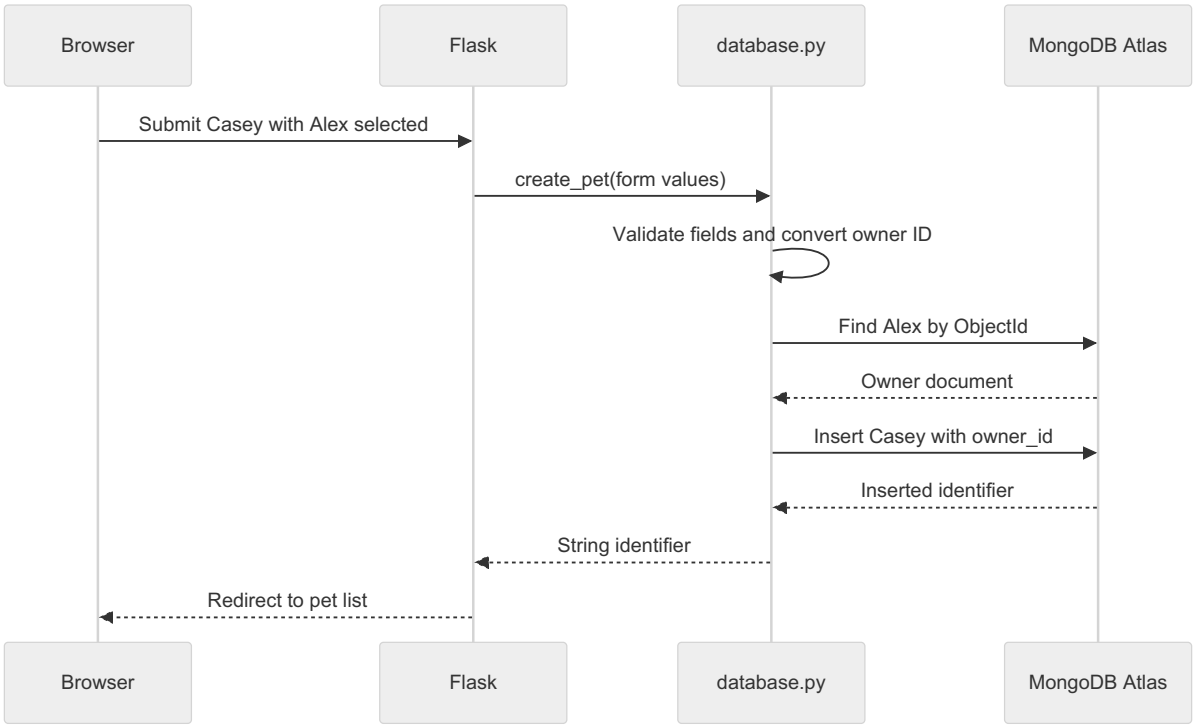
```
def create_pet(data):
    pet = _normalize_pet_data(data)
    result = pets_collection.insert_one(pet)
    return str(result.inserted_id)
```

The layer validates the proposed values, sends an insert through PyMongo, and returns the identifier as a string. Flask does not need a different form just because the write crosses a network.

Integrity Rules Still Belong in Our Layer

Moving to Atlas changes where the documents live. It does not replace our Python rules with relational constraints. We are still implementing in `database.py` the integrity rules that SQLite handled for the relational application.

The shared validation helper checks required text and rejects negative ages. It also finds the proposed owner before saving a pet. The owner deletion function looks for a dependent pet before allowing deletion. Create and update use the same validation rules.



Try This: Attempt to save Casey with age `-5`. The app should reject it, and the stored age should remain four. Then try to delete Alex while Casey still refers to Alex. The app should reject that deletion too.

These are checks in our layer, so carry out the exercise through the app. Editing directly in Data Explorer does not call `database.py`. MongoDB makes an individual document write atomic: the write happens as a unit. A separate owner lookup followed by a pet insertion is still two operations, not one automatic transaction. (10)

ACID Transactions and Eventual Consistency

MongoDB supports ACID transactions: Atomicity, Consistency, Isolation, and Durability. It can group changes across documents and collections into a transaction. The guarantees depend on the transaction's read and write concern settings. Our Pets code does not start such a transaction; its owner lookup and pet insertion remain separate operations. (11)

A MongoDB replica set keeps copies of data on several servers. Writes go to the primary, and secondary servers apply those changes asynchronously. A read from a secondary can therefore return an older value. **Eventual consistency** means that, when new writes stop and replication can proceed, those copies converge. It does not promise that every read immediately sees the latest write. PyMongo reads from the primary by default; the supplied application does not request secondary reads. (12)

Setting or mechanism	What it controls
Read preference	Which replica-set member receives a read
Read concern	The visibility and consistency guarantees of that read
Write concern	The acknowledgement required before a write reports success
Transaction	Which operations must commit or roll back together

These are separate choices. Eventual consistency concerns agreement among copies over time. The consistency in ACID concerns preserving database rules. MongoDB offers transaction and replication controls rather than replacing all ACID guarantees with eventual consistency. A majority acknowledgement also does not mean every secondary is already current. (11)(12)(13)

For example, Casey's age might already be four on the primary while a secondary still shows three. That is different from saving a pet whose owner does not exist. The first concerns the freshness of a replica; the second concerns an application integrity rule.

Moving Existing Data Is a Separate Job

We moved the application by changing its storage connection. A **data migration** would copy existing records into the new system while preserving their meaning and relationships. The local Mongita folder remains local; Atlas cannot read it just because we supplied a connection string.

For the class exercise, creating Alex and Casey gives us a small, visible check of the new connection. To migrate a larger Pets dataset, stop writes to the source while making a consistent copy. Read the owner documents first and then the pets. Preserve each `_id` and each pet's `owner_id` as `ObjectId` values when inserting into the destination. Generating new owner identifiers would require translating every corresponding pet reference.

After a copy, compare owner and pet counts and check that every destination pet refers to a destination owner. Open several pets through the application as well. Matching counts alone cannot establish that the relationships survived. The connection demonstration in this chapter checks access and counts; it is not a migration program.

When the Connection Does Not Work

Separate a missing local setting from authentication, permissions, and network access. Atlas's troubleshooting guide treats these as different causes, even when the first visible symptom is simply a failed connection. (14)

Symptom	Check next
`Set MONGODB_URI` message	Set the variable in the terminal that starts Python
Authentication failure	Database username, password, and percent-encoding
Timeout selecting a server	Deployment readiness and the application's allowed IP address
Name lookup failure	Copied hostname and network name resolution
Not authorized to read or write `pets`	Database user's role on the `pets` database
Connection works, but list is empty	Selected deployment and database; local data was not copied

An empty collection is a successful query with no matching documents. It is different from a failed connection. Run `access-atlas-demo.py` first to keep Flask and HTML rendering out of the initial diagnosis.

Practice

1. Run the connection demonstration before creating records and again after creating Alex and Casey. Record the counts and explain which operations changed them.
2. Create Blair, reassign Casey to Blair, and inspect Casey in Data Explorer. Confirm that `owner_id` changed while Casey's `_id` stayed the same. Delete Alex through the app after confirming no pets still refer to Alex.
3. Close the terminal running Flask. Open another, activate the environment, set `MONGODB_URI`, and start the app again. Confirm that Casey is still present. Explain which settings were local to the terminal and which data was stored remotely.
4. Create two owners with the same name. Give each a different pet. Use the identifiers in Data Explorer to establish which pet belongs to which owner. Explain why matching names are not enough.
5. In a second terminal, run the connection demonstration against the same deployment while Flask is running. Compare its counts with the app's lists. Both programs should see the same database.

Considerations

- Which parts of this move were made easier by having a database abstraction layer? Which connection details still required new work?
- Why should a program that only reads reports have different database privileges from the Pets application?
- If another program writes directly to these collections, where must the shared integrity rules be enforced?
- What could happen if one client deletes an owner between another client's owner lookup and pet insertion? Which operations would need coordination?
- When migrating records, why are equal counts weaker evidence than checking identifiers and references too?

References

1. Wikipedia contributors. “Cloud database.” https://en.wikipedia.org/wiki/Cloud_database
2. MongoDB. “Create an Atlas Account.” Atlas documentation. https://www.mongodb.com/docs/atlas/tutorial/create-atlas-account/
3. MongoDB. “Manage Project Access.” Atlas documentation. https://www.mongodb.com/docs/atlas/tutorial/manage-projects/
4. MongoDB. “Deploy a Free Cluster.” Atlas documentation. https://www.mongodb.com/docs/atlas/tutorial/deploy-free-tier-cluster/
5. MongoDB. “Configure Database Users.” Atlas documentation. https://www.mongodb.com/docs/atlas/security-add-mongodb-users/
6. MongoDB. “Configure IP Access List Entries.” Atlas documentation. https://www.mongodb.com/docs/atlas/security/ip-access-list/
7. MongoDB. “Connect to a Cluster via Client Libraries.” Atlas documentation. https://www.mongodb.com/docs/atlas/driver-connection/
8. MongoDB. “Choose a Connection Target.” PyMongo documentation. https://www.mongodb.com/docs/languages/python/pymongo-driver/current/connect/connection-targets/
9. MongoDB. “Databases and Collections in MongoDB.” MongoDB Manual. https://www.mongodb.com/docs/manual/core/databases-and-collections/
10. MongoDB. “Atomicity and Transactions.” MongoDB Manual. https://www.mongodb.com/docs/manual/core/write-operations-atomicity/
11. MongoDB. “Transactions.” MongoDB Manual. https://www.mongodb.com/docs/manual/core/transactions/
12. MongoDB. “Read Preference.” MongoDB Manual. https://www.mongodb.com/docs/manual/core/read-preference/
13. MongoDB. “Read Isolation, Consistency, and Recency.” MongoDB Manual. https://www.mongodb.com/docs/manual/core/read-isolation-consistency-recency/
14. MongoDB. “Troubleshoot Connection Issues.” Atlas documentation. https://www.mongodb.com/docs/atlas/troubleshoot-connection/